

Redes TCP/IP - Capa de Transporte

Redes de Comunicación, Manipulación y Telemedicina

Departamento de Ingeniería de Sistemas y Automática



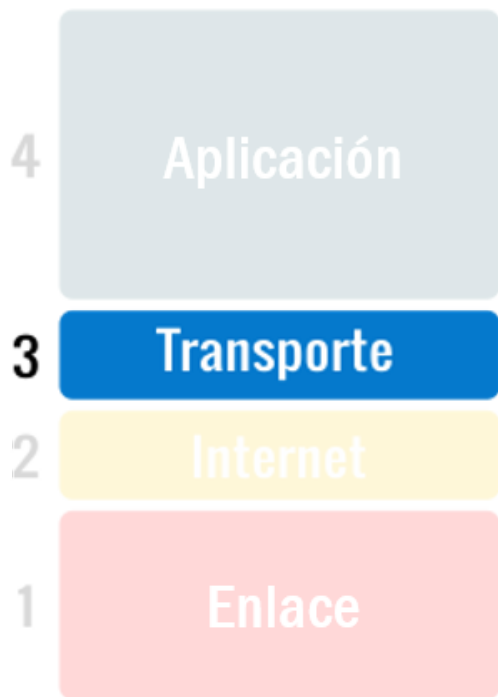
Escola de
Enxeñaría
Industrial

Universidade de Vigo

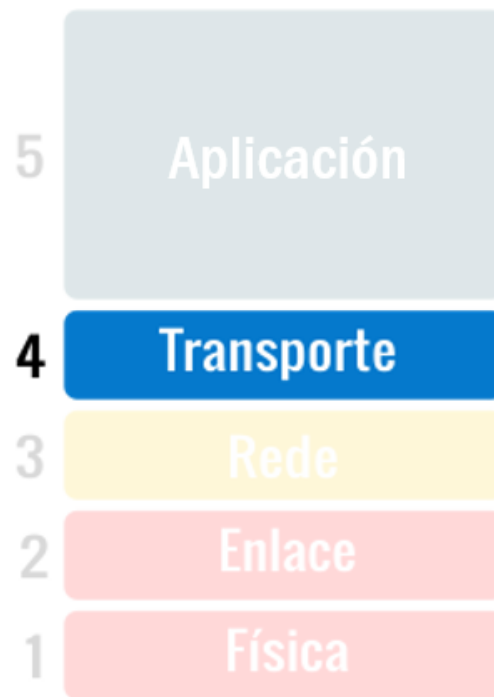
Contenidos

- Introducción a la capa de transporte
- UDP
- TCP
- Control de congestión

Modelo TCP/IP



**Modelo de Referencia TCP/IP
4 capas**



**Modelo de Referencia TCP/IP
5 capas**

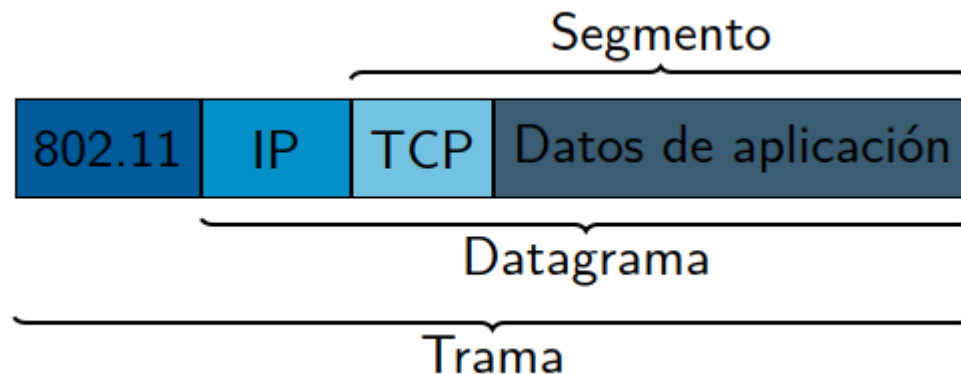
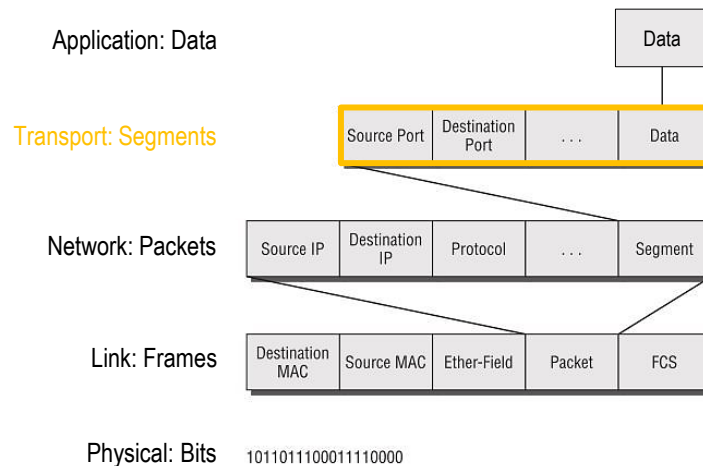
Introducción

La capa de transporte es un componente fundamental del modelo de red que se encarga de garantizar la entrega fiable de datos entre aplicaciones que se ejecutan en diferentes dispositivos. A continuación se detallan los aspectos clave de esta capa.

Conectividad Extremo a Extremo. La capa de transporte se encarga de establecer, mantener y finalizar conexiones entre los extremos de la comunicación.

Transporte de Datos. La capa de transporte utiliza estructuras de datos denominadas segmentos para encapsular la información que se intercambia entre las aplicaciones.

Encapsulación de Datos. En la capa de transporte comienza la encapsulación de los datos producidos por la Capa de Aplicación. La encapsulación aumentará en cada capa sucesiva.



Servicios 1/2

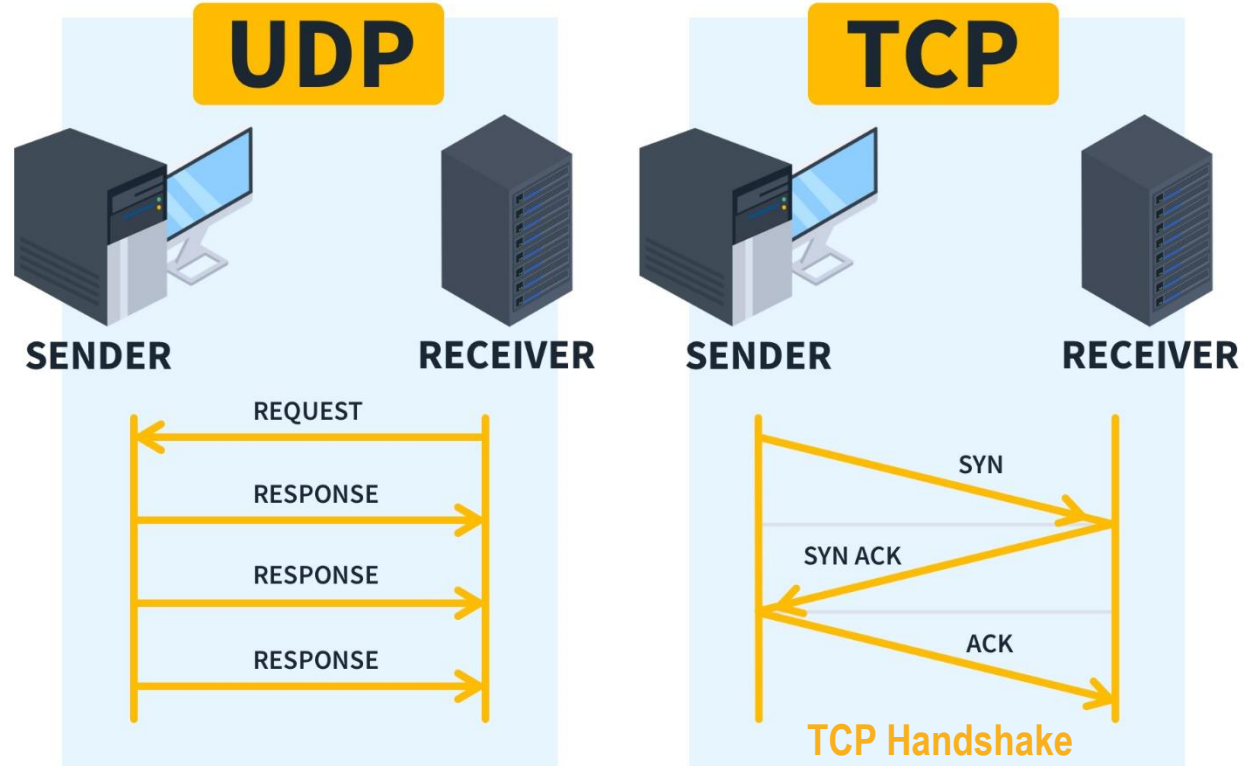
La capa de transporte en la red proporciona distintos tipos de servicios para la entrega de datos, siendo TCP (Transmission Control Protocol) y UDP (User Datagram Protocol) los dos protocolos más relevantes. A continuación se presentan las características y diferencias clave entre ambos:

TCP (Transmission Control Protocol)

Fiabilidad: TCP es un protocolo orientado a la conexión que garantiza la entrega fiable de datos. Esto significa que TCP se encarga de la corrección de errores, la retransmisión de paquetes perdidos y el reordenamiento de paquetes que llegan fuera de secuencia.

Flujo de Bytes: TCP considera los datos como un flujo de bytes, lo que permite que las aplicaciones envíen y reciban grandes cantidades de datos de manera continua. La capa de transporte segmenta estos datos en unidades más pequeñas (segmentos) para su envío.

Control de Flujo y Congestión: TCP implementa mecanismos de control de flujo para evitar la saturación de la red, y mecanismos de control de congestión para detectar y reaccionar ante condiciones de congestión.



Fuente: What are Transport Layer Protocols? <https://www.cb nuggets.com/blog/technology/networking/what-are-transport-layer-protocols>

UDP (User Datagram Protocol):

No Fiable: A diferencia de TCP, UDP es un protocolo sin conexión que no garantiza la entrega de los datos. No realiza correcciones de errores ni retransmisiones, lo que significa que los paquetes pueden perderse o llegar fuera de orden sin que esto se maneje automáticamente.

Datagramas: UDP envía datos en forma de datagramas individuales. Cada datagrama es independiente y contiene la información necesaria para que el receptor entienda a qué aplicación pertenece.

Menor Latencia: Debido a su naturaleza simple y sin conexión, UDP puede ofrecer menores tiempos de latencia. Esto lo hace ideal para aplicaciones en tiempo real como video streaming, juegos en línea y VoIP, donde la rapidez es más crítica que la fiabilidad.

La elección entre TCP y UDP depende de los requisitos específicos de la aplicación. Si se necesita una entrega fiable y ordenada de datos, TCP es la mejor opción. Sin embargo, si la velocidad y la eficiencia son más importantes y se puede tolerar cierta pérdida de datos, UDP puede ser el protocolo más adecuado.

Servicios 2/2

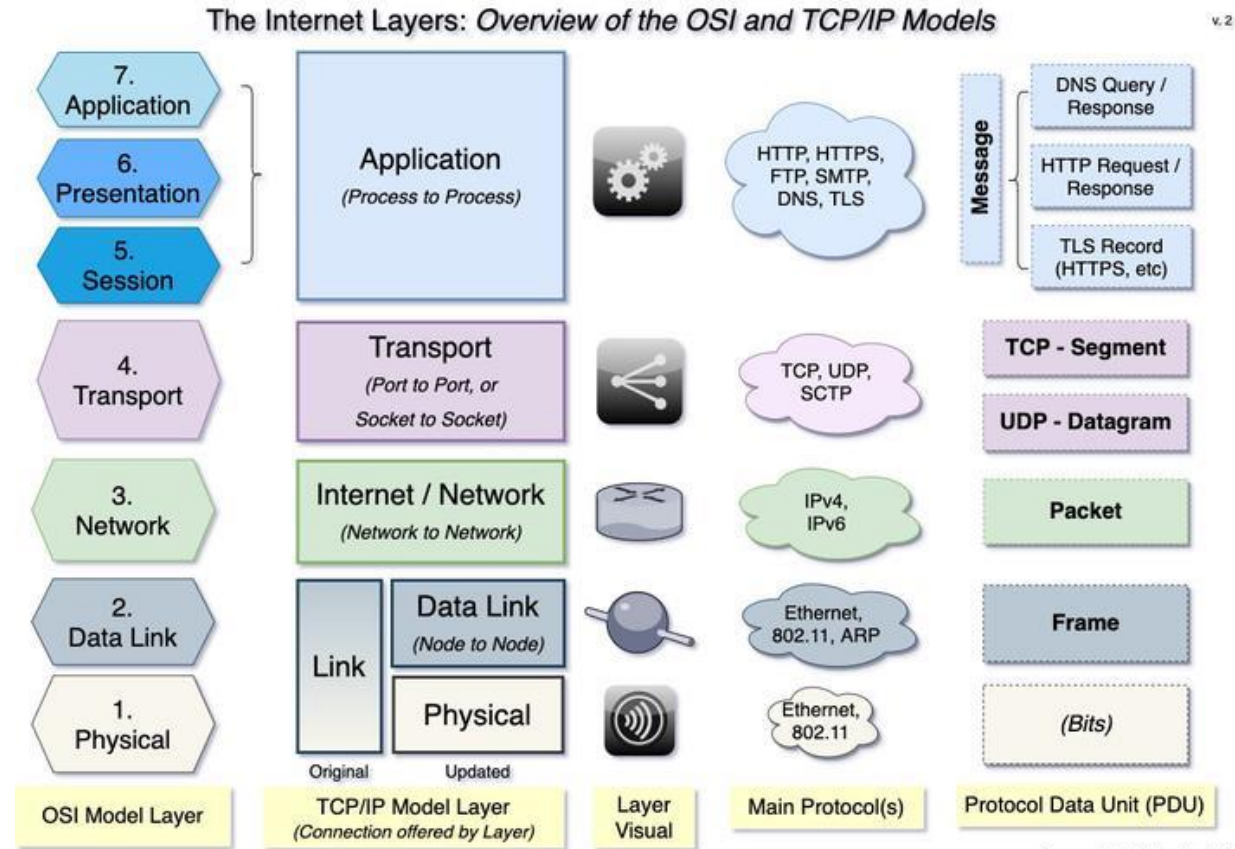
Característica	TCP	UDP
Fiabilidad	Sí (entrega garantizada y ordenada)	No (posibilidad de pérdida y desorden)
Conexiones	Orientado a conexión	Sin conexión
Tipo de mensaje	Flujos de bytes	Datagramas
Longitud de contenido	Arbitraria	Tamaño limitado del mensaje
Control de flujo	Sí (coordina transmisor y receptor)	No (transmisión independiente)
Control de congestión	Sí (ajusta según el estado de la red)	No (envío independiente del estado de la red)

Puertos

Un puerto es un número entero de 16 bits. Este número es capaz de diferenciar un total de 65536 puertos diferentes, numerados del 0 al 65535. Gracias a los puertos, la capa de transporte puede multiplexar distintas aplicaciones sobre una misma interfaz de red. Esto significa que múltiples conexiones pueden ser gestionadas simultáneamente, permitiendo que varias aplicaciones se comuniquen a través de una sola dirección IP.

En la capa de red (NAT), los puertos se usan como una técnica para facilitar el mapeo de direcciones y gestionar múltiples conexiones simultáneas desde una red interna hacia una red externa, pero no están directamente relacionados con el transporte de datos de aplicaciones.

En la capa de transporte (socket), los puertos son fundamentales para la identificación de servicios o aplicaciones que necesitan comunicarse entre dispositivos. Son usados para encaminar correctamente los datos de las aplicaciones al servicio adecuado.

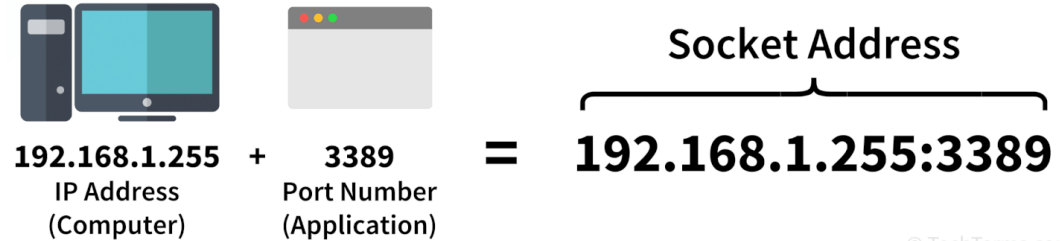


Sockets

Cada proceso de la capa de aplicación se identifica mediante una tupla que incluye la dirección IP del dispositivo y el número de puerto. A esta tupla, se le llama socket.

Aquí, los números de puerto permiten que un mismo dispositivo pueda manejar múltiples conexiones simultáneas a diferentes servicios o aplicaciones. En resumen, los puertos en la capa de transporte son usados para distinguir entre las diferentes aplicaciones que están comunicándose a través de una red.

Los servidores a menudo se asocian a puertos bien conocidos (well-known ports). Estos son puertos estandarizados que suelen estar asociados a servicios específicos. Están en el rango de 0 a 1023. La asignación de puertos en este rango generalmente requiere privilegios administrativos. Algunos ejemplos de puertos bien conocidos incluyen: 80 (HTTP), 443 (HTTPS) y 21 (FTP).



© TechTerms.com

Fuente: Socket <https://techterms.com/definition/socket>

Puerto	Protocolo	Uso
20,21	FTP	Transferencia de archivos
22	SSH	Acceso remoto
25	SMTP	Correo electrónico
80	HTTP	<i>World Wide Web</i>
110	POP3	Acceso remoto al correo electrónico
143	IMAP	Acceso remoto al correo electrónico
443	HTTPS	<i>Web segura</i>
543	RTSP	Control de reproducción multimedia
631	IPP	Compartición de impresoras

User Datagram Protocol (UDP)

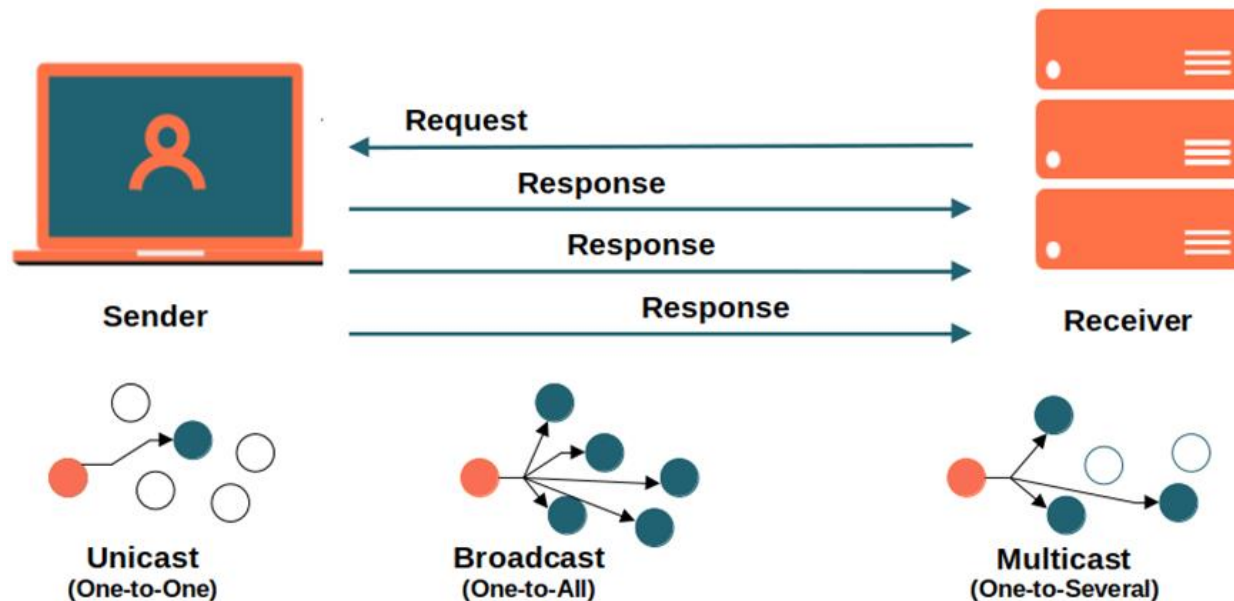
El Protocolo de Datagramas de Usuario (UDP) se utiliza para enviar mensajes en redes de comunicación. David P. Reed desarrolló el protocolo UDP en 1980. Está definido en el RFC 768 y forma parte del protocolo TCP/IP, por lo que es un estándar en Internet. El protocolo UDP permite a las aplicaciones de los ordenadores enviar mensajes en forma de datagramas de una máquina a otra a través de la red de Protocolo de Internet (IP). Este protocolo es utilizado por aplicaciones que no requieren flujos de datos garantizados y necesitan transmitir información en tiempo real.

Un ejemplo de aplicación que utiliza UDP es la voz sobre IP (VoIP). La VoIP requiere una transmisión de audio en tiempo real. Otros ejemplos incluyen el DNS (Sistema de Nombres de Dominio), que utiliza UDP para enviar mensajes de resolución de nombres de dominio, y el DHCP (Protocolo de Configuración Dinámica de Host), que utiliza UDP para enviar mensajes de configuración de red.

Si una aplicación requiere flujo de datos garantizado a al utilizar UDP, debe implementar mecanismos ARQ adicionales a nivel de aplicación para garantizar la entrega de los paquetes.

Introducción UDP

How User Datagram Protocol (UDP) Works



La longitud del datagrama de UDP puede ser de hasta 64 kilobytes (kB). Una de las características clave de la cabecera de UDP es que utiliza puertos para identificar los procesos de las aplicaciones que transmiten y reciben datos, lo que se conoce como multiplexación. La cabecera está formado por los siguientes campos:

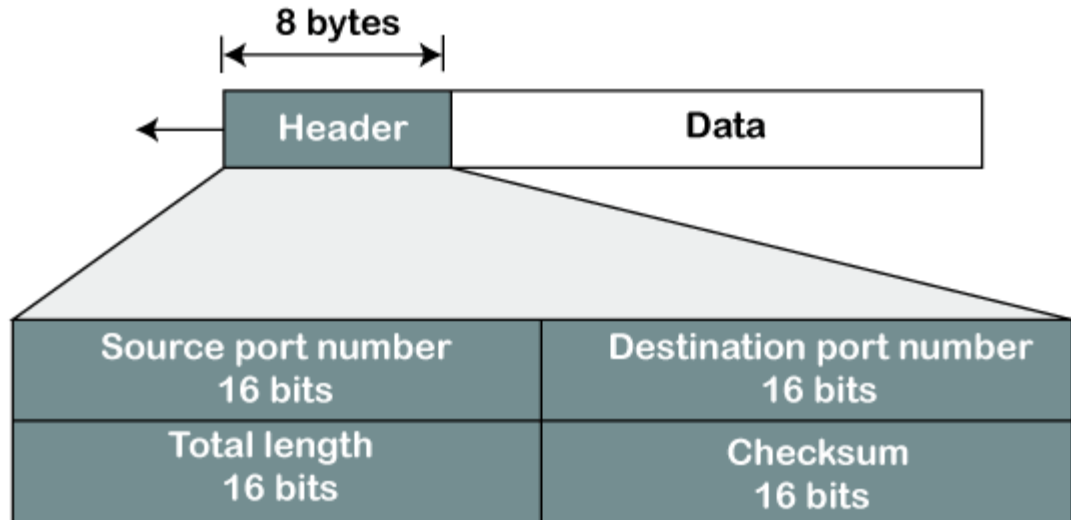
Puerto de origen (16 bits) y puerto de destino (16 bits). El puerto de origen y destino identifican el servicio a nivel de aplicación.

Longitud (16 bits). Es un campo de 16 bits que especifica la longitud total del paquete UDP, incluyendo también la cabecera. El valor mínimo sería de 8 bytes, ya que el tamaño de la cabecera es de 8 bytes.

Checksum (16 bits). Es un campo de 16 bits y es opcional. Este campo de verificación comprueba si la información es precisa o no, ya que existe la posibilidad de que se corrompa durante la transmisión. Es un campo opcional: depende de la aplicación decidir si desea escribir el checksum o no. Si no se desea escribir el checksum, los 16 bits se establecen en cero. En UDP, el campo de checksum se aplica a todo el paquete (cabecera y datos). Sin embargo, en IP, el campo de checksum solo se aplica a la cabecera.

Cabecera UDP

UDP Header Format

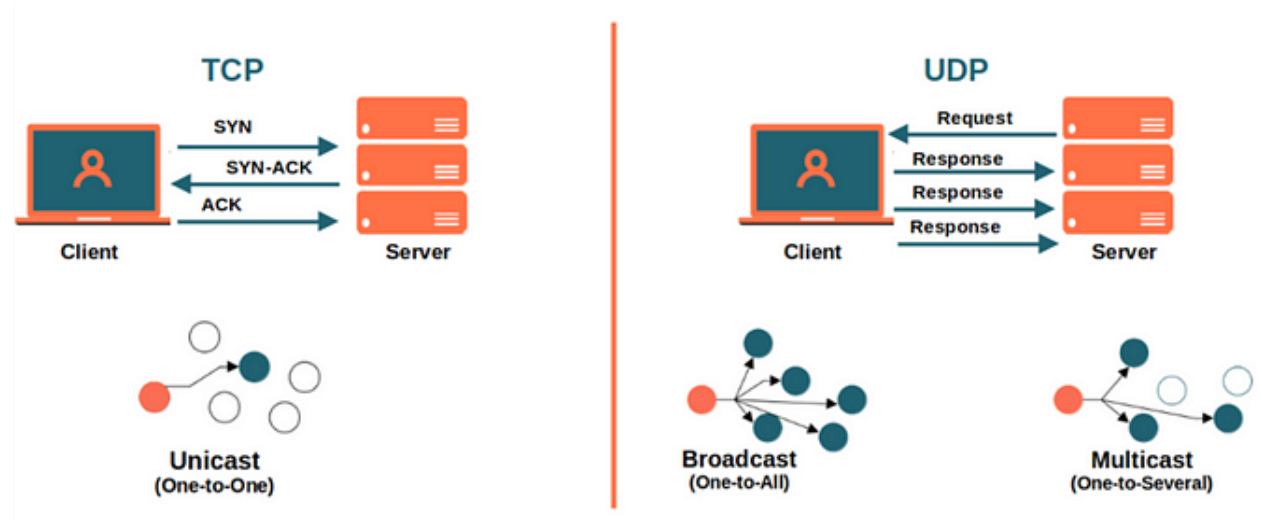


Transmission Control Protocol (TCP)

La mayoría del contenido en Internet utiliza el Protocolo de Control de Transmisión (Transmission Control Protocol, TCP) como protocolo de transporte. TCP es un protocolo orientado a conexión. Necesita establecer una conexión entre las partes que se van a comunicar antes de que puedan intercambiar datos. Una vez establecida la conexión, las partes pueden intercambiar flujos de datos de manera bidireccional.

TCP ofrece una serie de funciones importantes a las capas superiores, incluyendo la entrega fiable y ordenada de datos, el control de flujo y el control de congestión. La entrega fiable y ordenada de datos garantiza que los paquetes de datos sean entregados en el orden correcto y sin errores. El control de flujo regula la velocidad a la que se envían los datos para evitar sobrecargar la red.

Introducción TCP

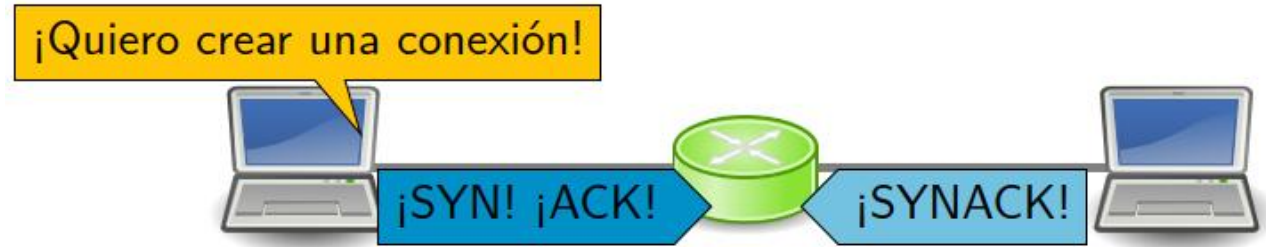


A diferencia de UDP, TCP requiere del establecimiento de una conexión entre las partes en comunicación antes de que puedan intercambiar datos. Este proceso es fundamental para garantizar la entrega fiable y ordenada de los datos.

Antes de comenzar la transferencia de datos, el receptor y el transmisor deben estar listos y acordar una serie de parámetros importantes. Por ejemplo, deben acordar el máximo tamaño de segmento (Maximum Segment Size, MSS) que se utilizará para la transferencia de datos. Este proceso de acordar parámetros se conoce como señalización.

El mecanismo de establecimiento de la conexión en TCP se denomina Three-Way Handshake por ser un proceso de tres pasos.

Establecimiento conexión TCP 1/2



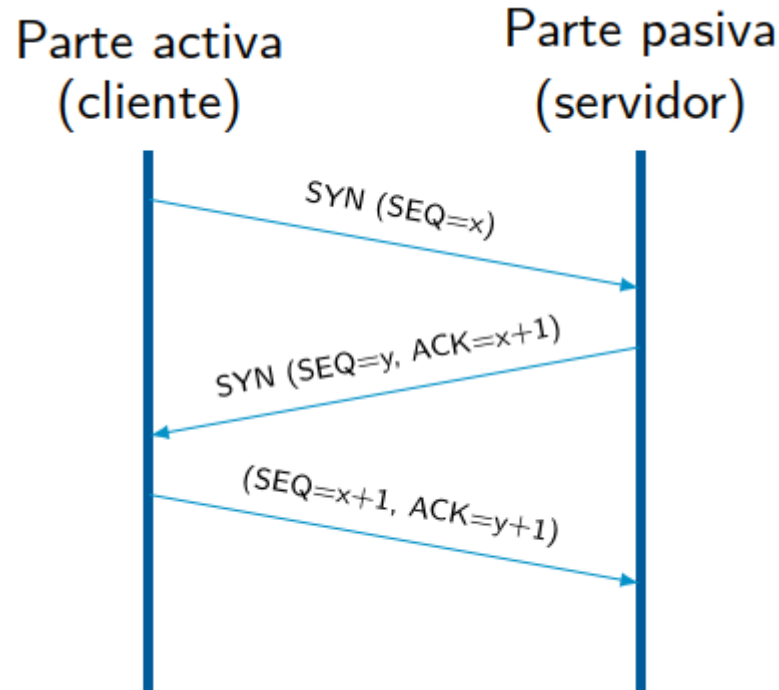
El Three-Way Handshake es un mecanismo utilizado en TCP para establecer conexiones. Durante el Three-Way Handshake, cada lado de la conexión le indica al otro un número inicial de secuencia (Initial Sequence Number, ISN) que se utilizará para enviar y recibir datos. El ISN se envía en un segmento de sincronización (SYN) y se hace eco con un mensaje de asentimiento (ACK). Esto se hace para evitar envío de datos duplicados debido a retrasos en la transmisión.

El Three-Way Handshake consta de tres pasos:

1. El cliente envía un segmento SYN(x) al servidor, que incluye el número inicial de secuencia del cliente "x".
2. El servidor responde con un segmento SYN(y), ACK(x+1), que incluye el número inicial de secuencia del servidor "y" y un mensaje de asentimiento para el número de secuencia del cliente "x+1".
3. El cliente responde con un segmento ACK(y+1), que confirma la recepción del número de secuencia del servidor.

Una vez establecida la conexión, los números de secuencia y de asentimiento se tienen en cuenta para futuros segmentos. En caso de perderse segmentos, la numeración facilita la retransmisión puntual de los datos perdidos.

Establecimiento conexión TCP 2/2



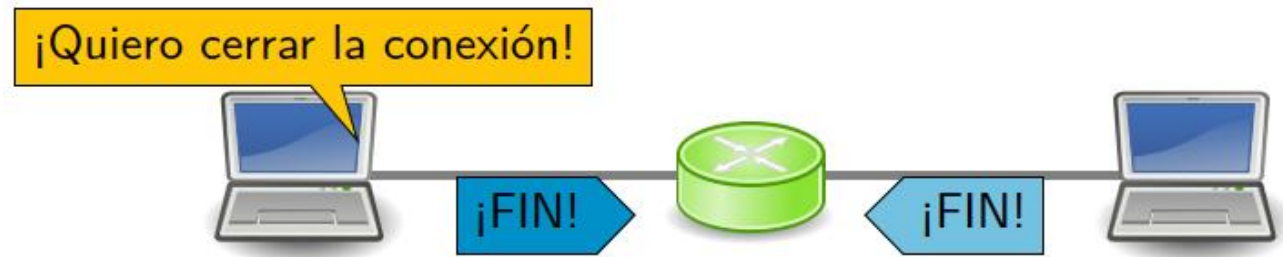
La liberación de la conexión en TCP se produce cuando ambas partes de la conexión han finalizado la transmisión de datos. En este momento, se entregan todos los datos pendientes y se "cuelga" la conexión.

El problema clave en la liberación de la conexión es proporcionar fiabilidad mientras se produce el cierre. Para solucionar este problema, TCP utiliza un cierre "simétrico" en el cual ambas partes finalizan la conexión de forma independiente. .

Además, TCP espera un tiempo largo (aproximadamente 30 segundos) antes de completar el cierre de la conexión. Esto se hace para asegurarse de que no haya mensajes perdidos que hayan estado circulando por la red y que puedan llegar después del cierre de la conexión. En caso de que lleguen mensajes perdidos, se reenviará el mensaje de fin de conexión para reiniciar el proceso de liberación.

Este proceso de liberación de la conexión en TCP garantiza que la conexión se cierre de manera segura y eficiente, y que no haya problemas de datos perdidos o corruptos.

Liberación conexión TCP 1/2



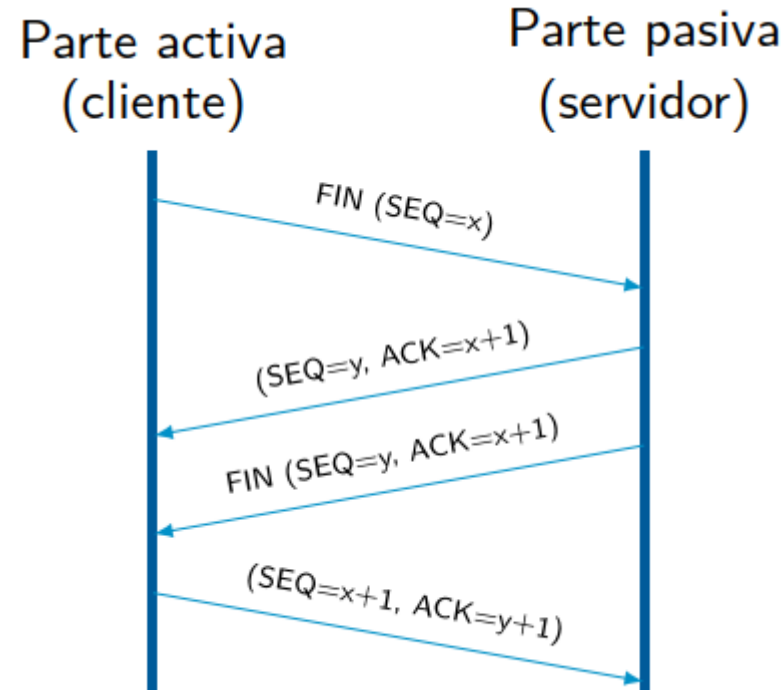
El cierre de conexión en TCP es un proceso que se utiliza para finalizar una conexión establecida entre dos dispositivos. Este proceso consta de dos pasos:

1. La parte activa de la conexión envía un mensaje $FIN(x)$ a la parte pasiva. Este mensaje indica que ya no tiene más datos que enviar. La parte pasiva asiente con un mensaje ACK , lo que confirma la recepción del mensaje FIN .
2. La parte pasiva envía un mensaje $FIN(y)$ a la parte activa, lo que indica que también ha finalizado la transmisión de datos. La parte activa asiente con un mensaje ACK .

En caso de que se pierdan mensajes FIN durante el cierre de la conexión, se retransmiten para garantizar que la conexión se cierre de manera correcta. Cada par FIN/ACK cierra las comunicaciones en ese sentido, lo que significa que la conexión se cierra en ambos sentidos.

Es importante destacar que el cierre de la conexión en TCP es un proceso bilateral, es decir, que requiere la participación de ambas partes de la conexión. Esto garantiza que la conexión se cierre de manera segura y eficiente.

Liberación conexión TCP 2/2



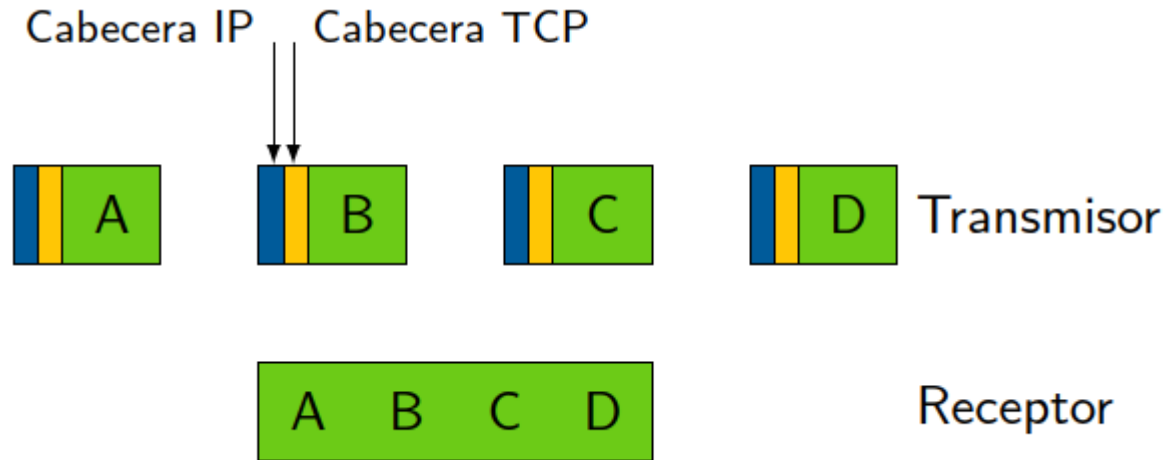
TCP - Fiabilidad

Fiabilidad

Para conseguir fiabilidad en la transmisión de datos, TCP utiliza mecanismos similares a los que se utilizan en el nivel de enlace. Uno de estos mecanismos es un esquema de ARQ (Automatic Repeat-Request) basado en el mecanismo de ventana deslizante. Esto permite a TCP detectar y retransmitir segmentos que se han perdido durante la transmisión.

Además, TCP utiliza números de secuencia para detectar pérdidas y ordenar los segmentos. Esto permite a TCP asegurarse de que los segmentos se entreguen en el orden correcto y detectar cualquier pérdida de segmentos.

Para retransmitir segmentos perdidos, TCP utiliza temporizadores. Si un segmento no es recibido dentro de un plazo determinado, TCP retransmite el segmento para asegurarse de que se entregue correctamente.



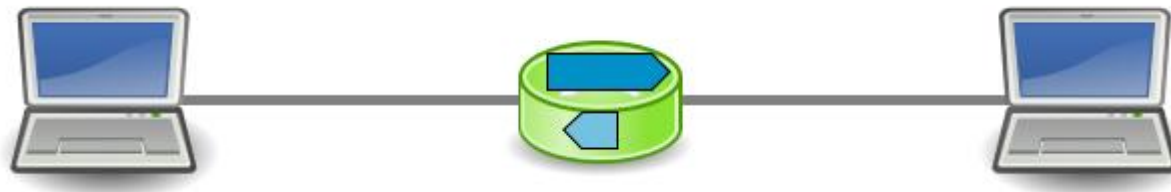
El control de flujo por Parada y Espera solo permite que un paquete esté pendiente de asentimiento en cualquier momento. Esto significa que solo se puede enviar un paquete por vez y que se debe esperar a que se reciba el asentimiento antes de enviar el siguiente paquete.

En redes con una latencia pequeña, como las redes LAN, este protocolo puede ser eficiente. En este caso, el tiempo de ida y vuelta (Round-trip Time, RTT) es pequeño. Este es el doble del tiempo de latencia D de enviar un mensaje de un extremo a otro. Sin embargo, en comunicaciones a través de Internet, donde la latencia es mucho mayor, este protocolo no es eficiente. En este caso, el RTT es mucho mayor: puede darse una gran latencia y una baja eficiencia en la transmisión de datos.

Supongamos una conexión con una latencia D de 50 ms. En parada y espera, solamente se puede enviar un paquete cada $2 \cdot D$. Por tanto, solamente se puede enviar 10 paquetes por segundo (pkt/s). Si el tamaño de paquete es 1250 B, esto produce una tasa aproximada de 100 kb/s. En una red con un ancho de banda de 1 Mb/s, sería una eficiencia del 10 %.

El protocolo de parada y espera solo requiere un bit para el número de secuencia

Parada y Espera

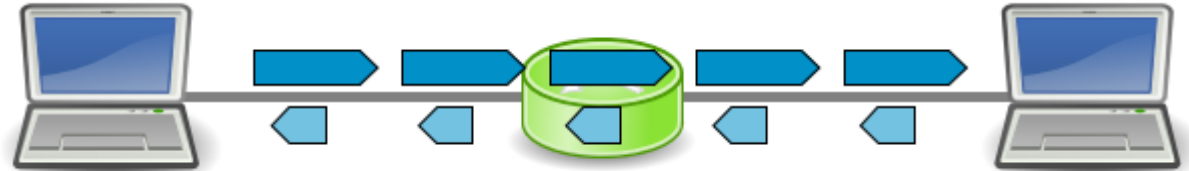


Ventana deslizante 1/3

El algoritmo de ventana deslizante es una generalización del algoritmo de parada y espera: se permite que W paquetes estén pendientes de asentimiento en cualquier momento. En este caso, se pueden enviar W paquetes por cada RTT. Recordar que el RTT es el doble de la latencia ($RTT = 2D$).

Este algoritmo es más eficiente para comunicaciones a través de Internet, donde la latencia es mayor. Al permitir que W paquetes estén pendientes de asentimiento, se puede aprovechar mejor la capacidad de la red y reducir la latencia.

Por ejemplo, sea $W = 10$ y la latencia $D = 50$ ms. La latencia de un único paquete limitaría a 10 pkt/s la comunicación. Sin embargo, al permitirse 10 de forma simultánea, la cifra aumenta a una tasa de 1 Mb/s. Si la conexión tuviera un ancho de banda de 1 Mb/s la latencia sería del 100 %. Si fuese de 10 Mb/s, la latencia sería del 10 %.



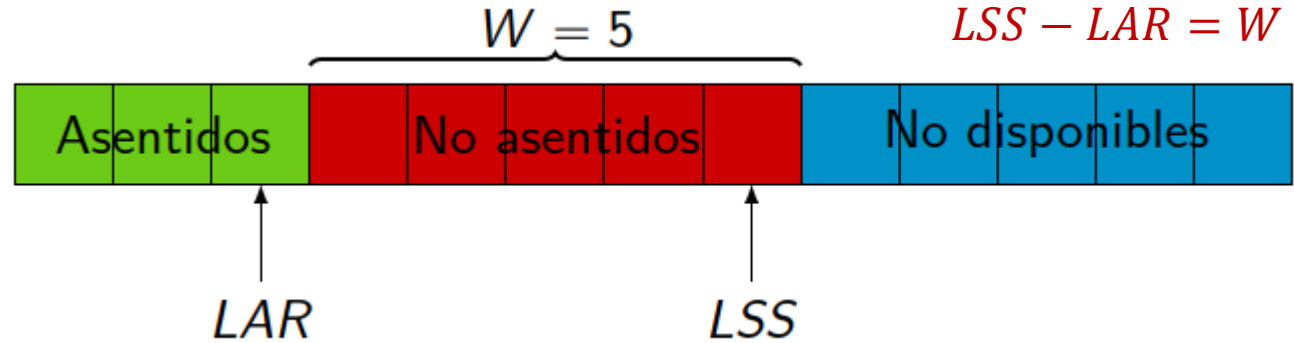
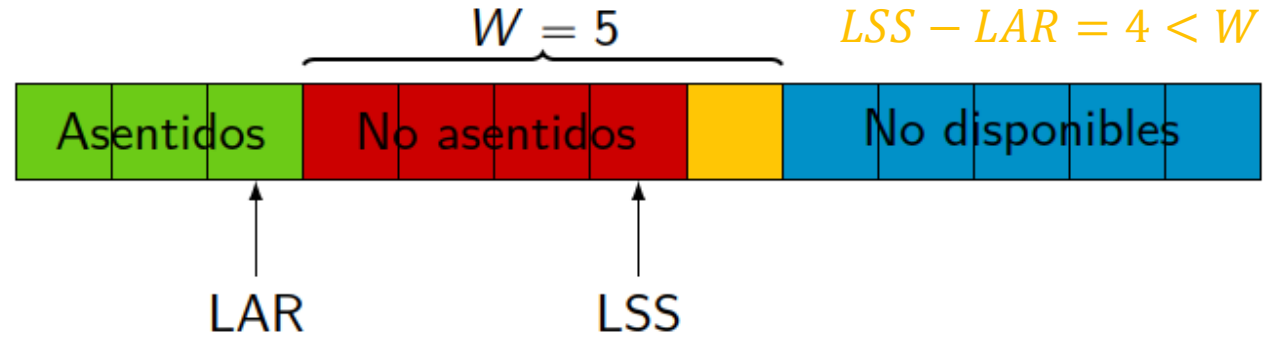
El transmisor almacena hasta W segmentos. Utiliza dos campos importantes para gestionar la ventana deslizante: el último segmento enviado (Last Segment Sent, LSS) y el último asentimiento recibido (Last Acknowledgement Received, LAR). La ventana deslizante se desliza cuando el transmisor recibe un nuevo asentimiento del receptor. Se transmite mientras $LSS - LAR \leq W$.

El proceso de envío de datos en el transmisor se puede describir de la siguiente manera:

1. El transmisor almacena hasta W segmentos de datos en su búfer.
2. El transporte acepta otro segmento desde la capa de aplicación y lo envía.
3. El transmisor envía el segmento y actualiza su variable LSS.
4. Llega un nuevo ACK del receptor y la ventana avanza.
5. El búfer se libera y el transmisor puede enviar un nuevo segmento.

En este ejemplo, el transmisor puede enviar un nuevo segmento porque la ventana deslizante se ha deslizado y el búfer se ha liberado. El transmisor puede enviar un nuevo segmento siempre y cuando la diferencia entre LSS y LAR sea menor o igual a W .

Ventana deslizante 2/3



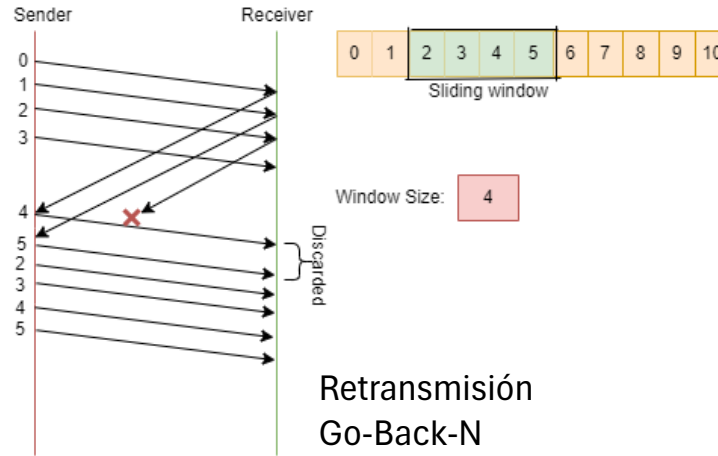
Ventana deslizante 3/3

En las ventanas deslizantes, las retransmisiones son necesarias para garantizar la entrega fiable de los segmentos de datos. Hay dos enfoques diferentes para manejar las retransmisiones, dependiendo del protocolo de ventana deslizante que se utilice.

En el caso de Go-Back-N, el transmisor utiliza un temporizador para detectar pérdidas de segmentos. Si el temporizador vence, el transmisor reenvía todos los segmentos almacenados desde LAR + 1 en adelante. Esto significa que se reenvían todos los segmentos que no han sido asentidos, lo que puede ser ineficiente en caso de pérdida de un solo segmento.

En el caso de la repetición selectiva, el transmisor utiliza un temporizador por cada segmento no asentido para detectar pérdidas. Cuando vence el temporizador de un segmento, se reenvía solo ese segmento. Esto significa que se envían menos segmentos en caso de pérdida, lo que puede ser más eficiente que el enfoque de Go-Back-N.

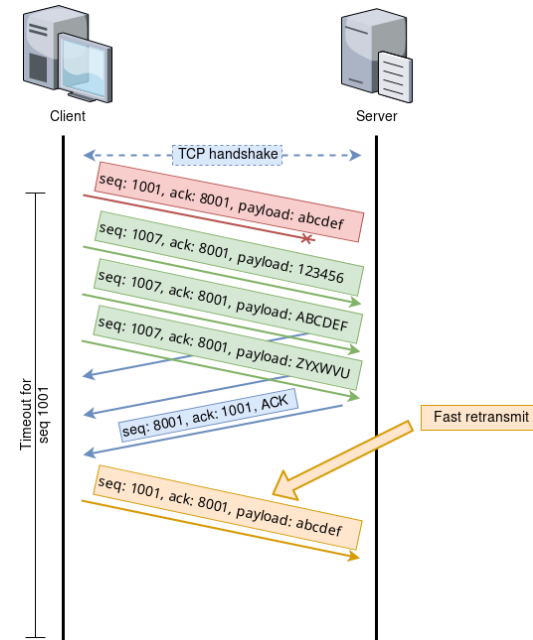
En TCP, los números de secuencia son utilizados para garantizar la fiabilidad de la transmisión de datos. A diferencia del protocolo de parada y espera, que solo requiere un bit para el número de secuencia, TCP requiere más de un bit para garantizar la fiabilidad.



Fuente: Go-Back-Na ARQ

<https://www.naukri.com/code360/library/go-back-n-arq-protocol>

Retransmisión Selectiva



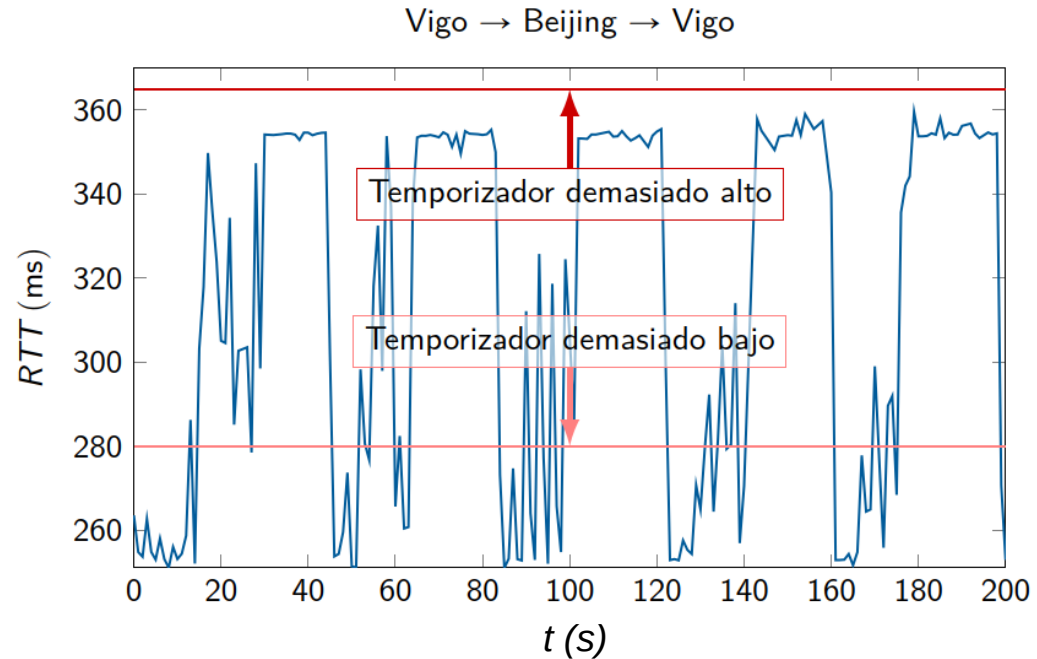
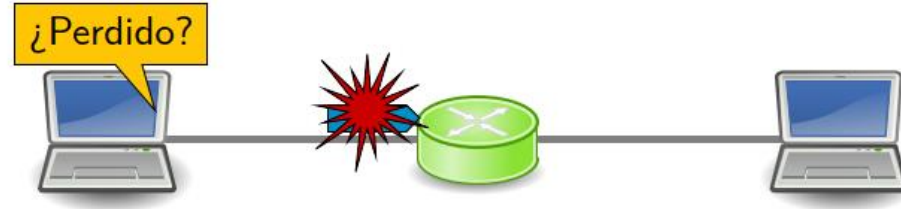
Fuente: Implement TCP Fast Retransmit <https://github.com/smoltcp-rs/smoltcp/issues/104>

Una de las estrategias para detectar una pérdida de datos es utilizar un temporizador. El temporizador se inicia cuando se envía un segmento de datos y se cancela cuando se recibe un asentimiento. Si el temporizador vence, se considera que el segmento se ha perdido y se retransmite.

Sin embargo, el temporizador debe ser "correcto". Si el temporizador es demasiado largo, se desperdicia capacidad de red, ya que se pueden enviar más segmentos de datos que no han sido asentidos. Por otro lado, si el temporizador es demasiado corto, se pueden producir retransmisiones innecesarias, lo que también desperdicia capacidad de red.

La pregunta es, ¿cuánto tiempo es el "correcto" para el temporizador? En una LAN (capa de enlace), es fácil establecer un temporizador, ya que el tiempo de ida y vuelta (Round-trip Time) es predecible. Sin embargo, en Internet (capa de transporte), el RTT es muy variable y puede variar desde unos pocos milisegundos hasta varios segundos. Esto hace que sea difícil establecer un temporizador "correcto" para detectar pérdidas de datos.

Retransmisiones vencimiento temporizador

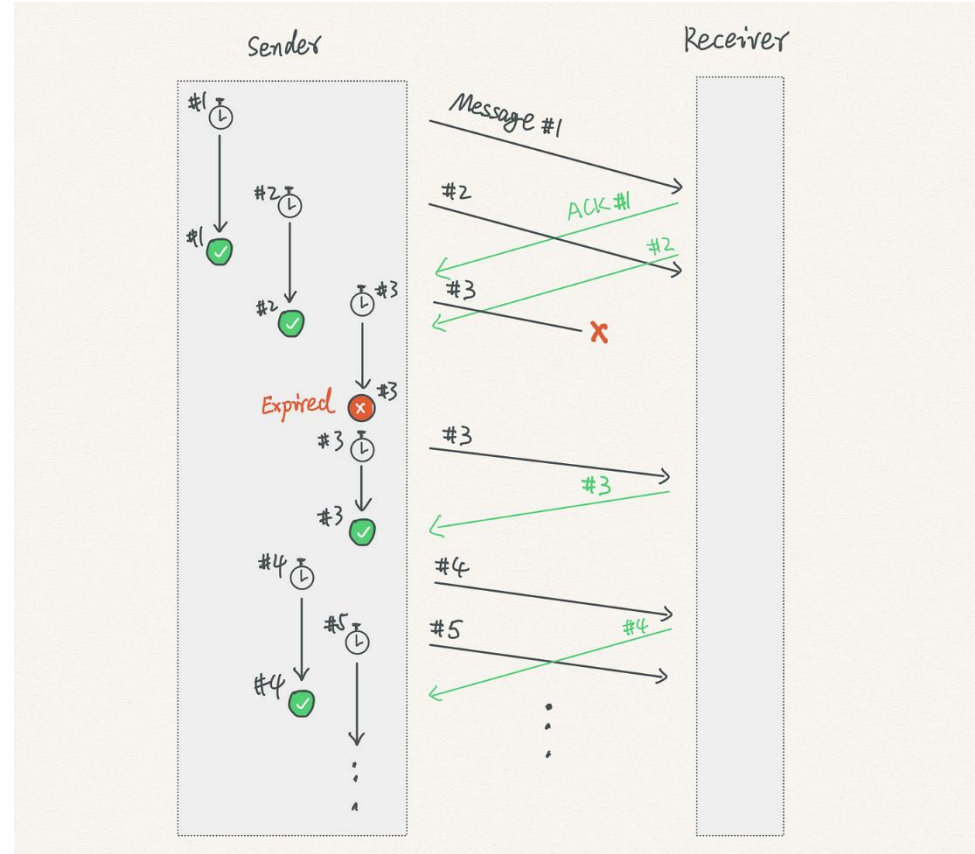


El mecanismo de temporizador adaptativo en TCP se especifica en el RFC 6298 y se basa en tomar muestras del RTT para estimar el temporizador de retransmisión. Este temporizador se denomina RTO (Retransmission Timeout).

Para implementar este mecanismo, el emisor TCP debe mantener dos variables de estado: una estimación del RTT, SRTT (smoothed RTT); una estimación de la variación del RTT, RTTVAR (RTT variation). Se utilizan medias móviles exponencialmente ponderadas para realizar la estimación de ambas variables. Esto significa que se asigna un peso mayor a las muestras más recientes del RTT y un peso menor a las muestras más antiguas.

El RFC (Request For Comments) que define el algoritmo recomienda una velocidad de adaptación de la estimación del RTT de $1/8$ y una velocidad de adaptación de la estimación de la variación del RTT de $1/4$.

Temporizador adaptativo



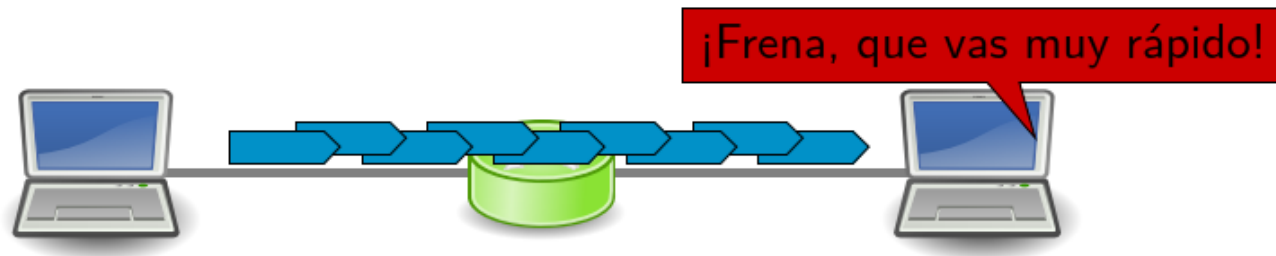
TCP - Control de Flujo

La ventana deslizante por sí sola no es suficiente para controlar el flujo de datos. Es necesario agregar un mecanismo de control de flujo que frene al transmisor cuando el receptor esté sobrecargado.

La ventana deslizante utiliza la serialización para mantener a la red ocupada. Esto significa que el transmisor envía datos en una secuencia continua, sin esperar a que el receptor confirme la recepción de cada paquete. Sin embargo, si el receptor está sobrecargado, no podrá procesar los datos a la misma velocidad que el transmisor los envía. En este caso, el receptor puede enviar un mensaje de control de flujo al transmisor para que reduzca la velocidad de transmisión.

El control de flujo se añade a la ventana deslizante mediante la utilización de un campo de control de flujo en la cabecera TCP. Este campo indica al transmisor la cantidad de datos que el receptor puede procesar en un momento determinado. Si el receptor está sobrecargado, puede reducir el valor de este campo para que el transmisor reduzca la velocidad de transmisión. De esta manera, se evita que el receptor se sobrecargue y se garantiza que la conexión sea estable y eficiente.

Introducción al control de flujo



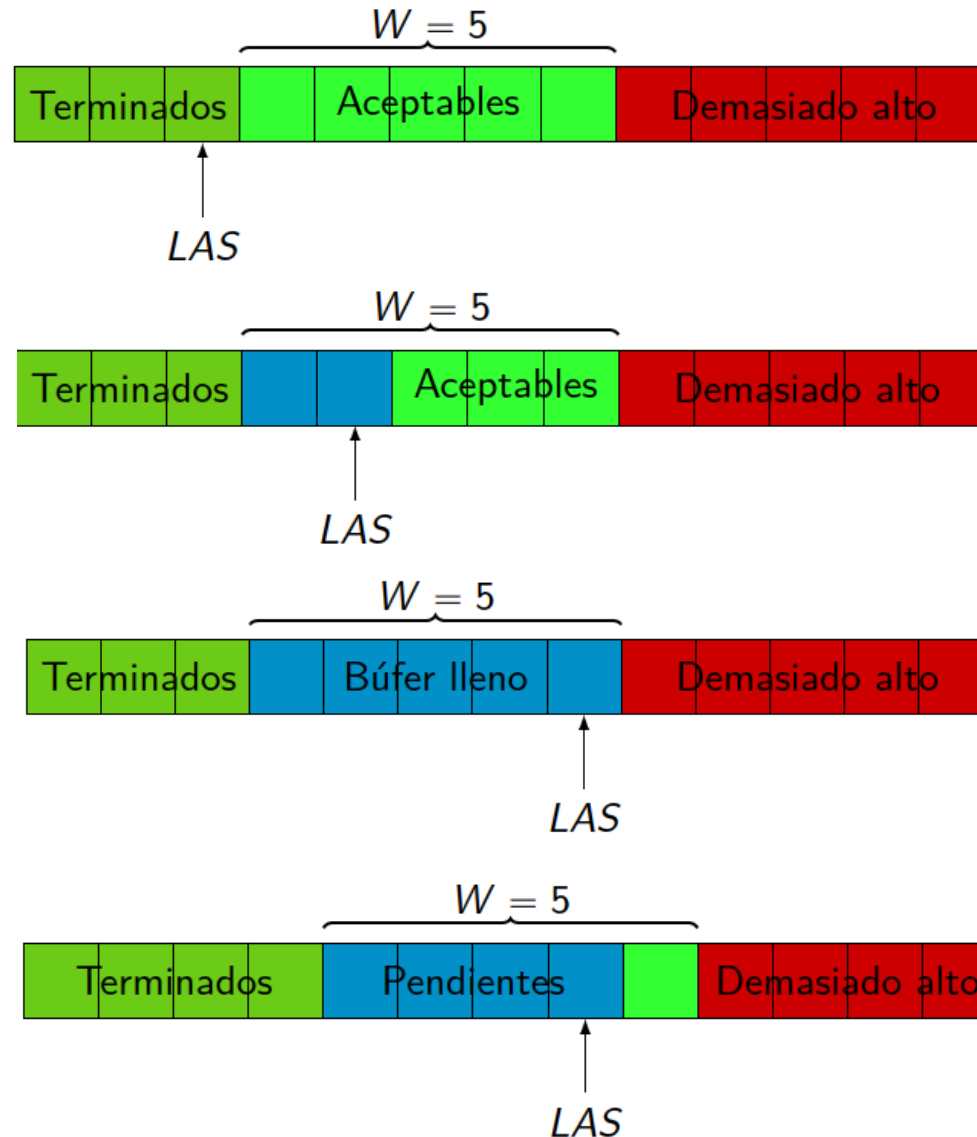
Receptor

Consideremos un receptor con un búfer de recepción de tamaño W . El receptor utiliza este búfer para almacenar los segmentos de datos que llegan de la red (#Aceptables). La capa de aplicación del receptor va sacando segmentos del búfer a medida que los puede procesar (#Terminados).

Supongamos que llegan dos segmentos al receptor, pero la capa de aplicación no los recoge de inmediato. En este caso, el último asentimiento enviado (Last Acknowledgement Sent, LAS) se incrementa, pero la ventana de recepción no se desliza. Esto significa que el receptor ha recibido los segmentos, pero no ha procesado aún.

Si llegan nuevos segmentos y la capa de aplicación no retira ninguno del búfer, el búfer se acabará llenando (#Búfer lleno). A partir de este momento, cualquier nuevo segmento que llegue será descartado hasta que el búfer se vacíe. Esto se debe a que el receptor no tiene espacio en el búfer para almacenar más segmentos.

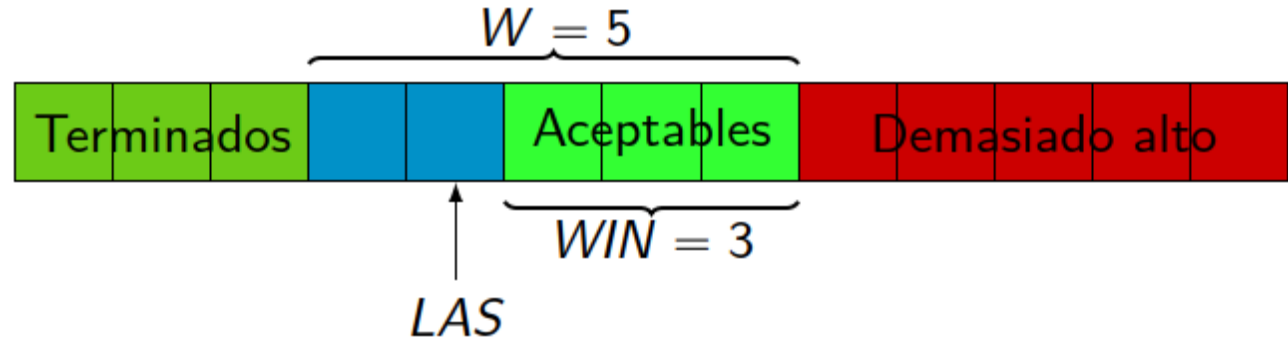
Sin embargo, si la capa de aplicación retira un segmento del búfer, se libera espacio en el búfer y el receptor podrá volver a almacenar segmentos. En este caso, la ventana de recepción se desliza, lo que significa que el receptor puede recibir nuevos segmentos y procesarlos.



Para evitar pérdidas de datos, el receptor puede informar al transmisor sobre el espacio disponible en su búfer de recepción. Esto se hace mediante el campo WIN (Window) en la cabecera TCP, que indica el número de huecos libres en el búfer (#Aceptables).

El transmisor utiliza el valor de WIN para determinar el tamaño de la ventana de recepción. Si el transmisor tiene una ventana de recepción (W) más grande que la ventana de control de flujo (WIN), utiliza el valor de WIN como el tamaño de la ventana efectivo. De esta manera, el transmisor no envía más datos de los que el receptor puede procesar.

Control de flujo



TCP – Cabeceras

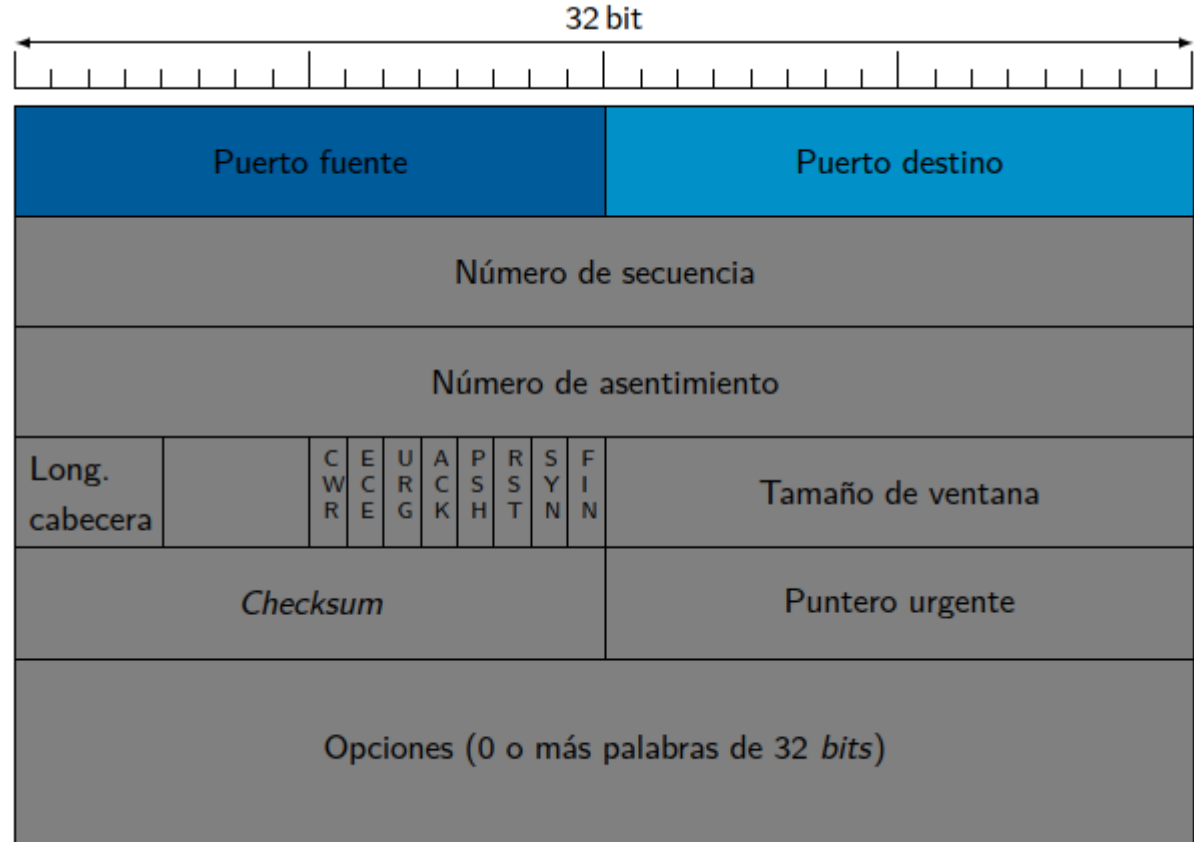
Cabecera TCP 1/2

La cabecera TCP es una estructura de datos que se utiliza para identificar y controlar las conexiones de red en el protocolo TCP. La cabecera TCP contiene varios campos que se utilizan para diferentes propósitos.

Los puertos (fuente y destino) son números de 16 bits que se utilizan para identificar a las aplicaciones que se están comunicando. Cada aplicación tiene un puerto asignado que se utiliza para enviar y recibir datos.

La ventana deslizante es un mecanismo de control de flujo que se utiliza para regular la cantidad de datos que se envían en una conexión de red. La ventana deslizante se basa en dos campos: SEQ y ACK. El campo SEQ indica la posición en el flujo del primer byte de datos que se está enviando, mientras que el campo ACK indica el siguiente byte que se espera recibir por parte del receptor.

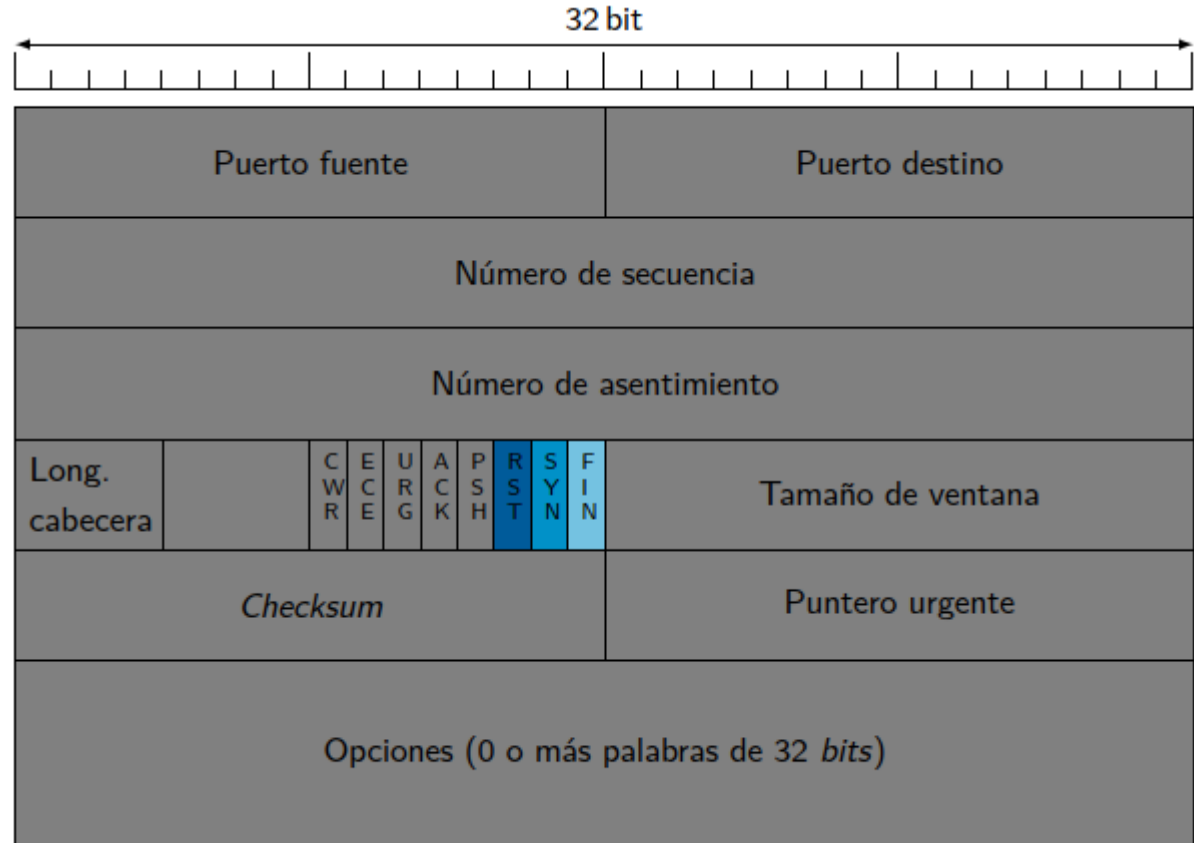
El piggybacking es una técnica que se utiliza para aprovechar un segmento de datos para enviar un ACK. Esto significa que se puede enviar un ACK junto con un segmento de datos, lo que reduce la cantidad de tráfico de red y mejora la eficiencia de la conexión.



Los flags SYN, FIN y RST se utilizan para el control de las conexiones. El flag SYN se utiliza para establecer una conexión, el flag FIN se utiliza para finalizar una conexión y el flag RST se utiliza para reiniciar una conexión.

Finalmente, el tamaño de ventana se utiliza para el control de flujo. El tamaño de ventana es relativo al último ACK que se recibió y se mide en bytes. Esto significa que el tamaño de ventana se ajusta dinámicamente según la cantidad de datos que se envían y se reciben en la conexión.

Cabecera TCP 2/2



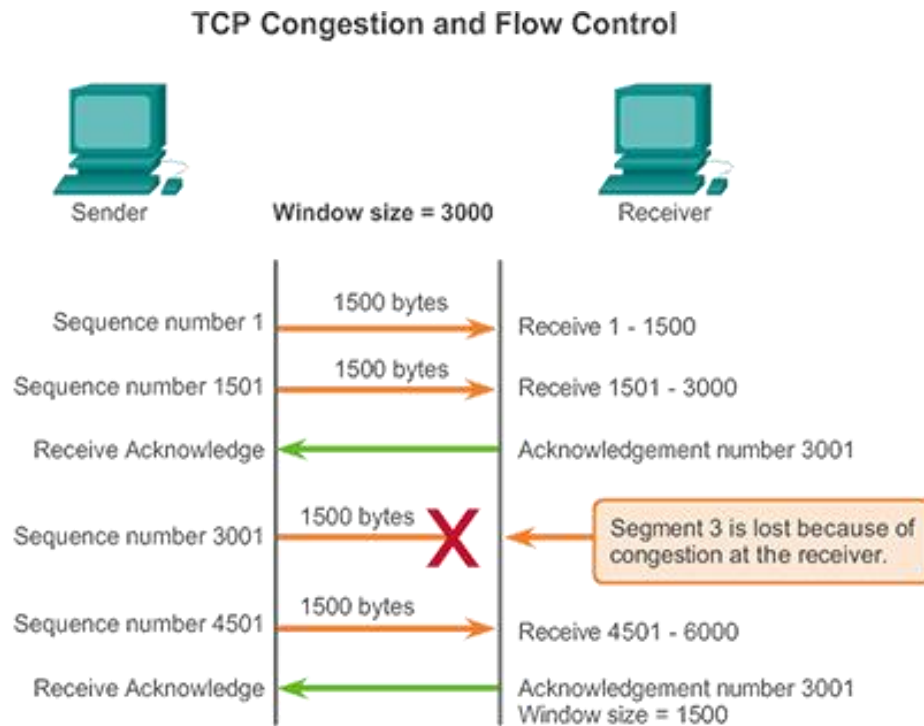
Control de Congestión

La congestión en las redes es un fenómeno que ocurre cuando la cantidad de datos que intenta atravesar la red supera la capacidad de sus dispositivos, como encaminadores y conmutadores. Estos dispositivos están equipados con búferes internos, que actúan como zonas de almacenamiento temporal para los paquetes de datos que están en tránsito.

Cuando la tasa de entrada de paquetes es mayor que la de salida, los búferes pueden llenarse rápidamente. En situaciones normales, estos búferes permiten que se gestionen picos temporales de tráfico. Sin embargo, si este desbalance persiste durante un período prolongado, los búferes se desbordarán y se producirá una congestión de la red.

Las consecuencias de esta congestión son significativas. A medida que los paquetes se acumulan, aumenta el retardo en la red (Round-Trip Time, RTT). Esto significa que aumenta el tiempo que tarda un paquete en ir y regresar. Además, la congestión provoca pérdidas de paquetes. A llenarse los búferes, los nuevos paquetes entrantes son descartados. El descarte puede resultar en la necesidad de retransmisiones y, por ende, en un uso aún mayor de los recursos de la red. Tanto el aumento del retardo como la pérdida de paquetes son indicadores claros de que una red puede estar experimentando congestión.

Naturaleza de la congestión



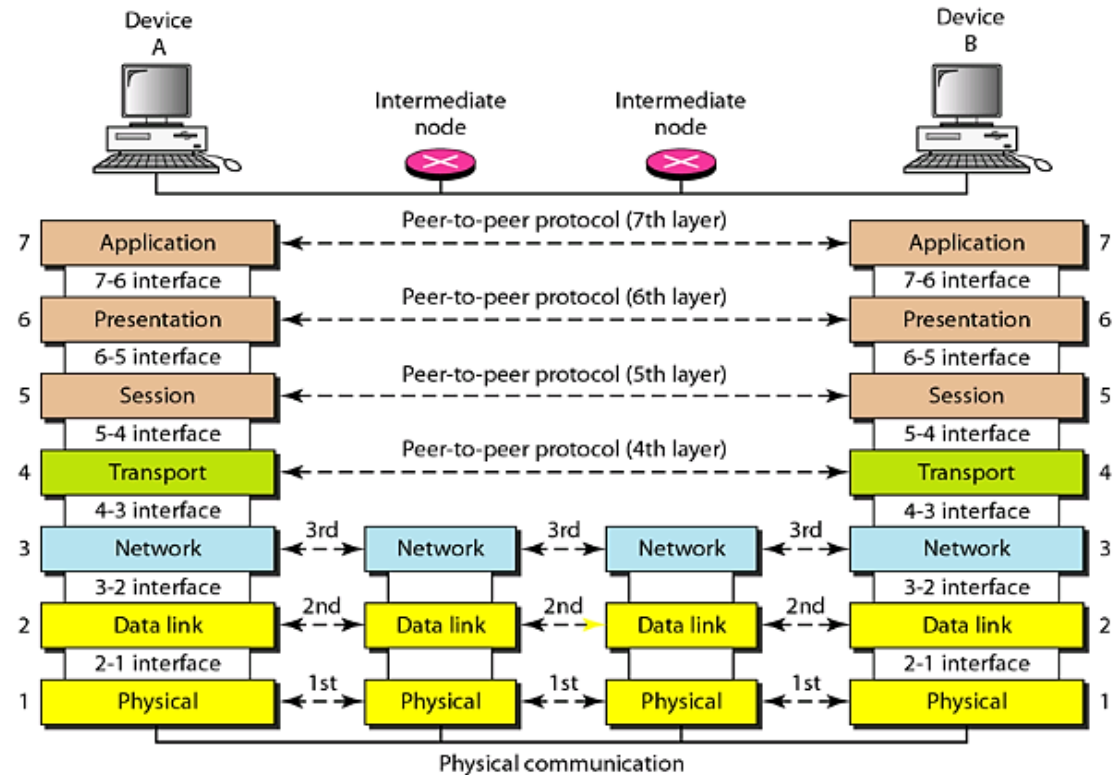
If segments are lost because of congestion, the receiver will acknowledge the last received sequential segment and reply with a reduced window size.

Es importante el uso de mecanismos de control de congestión para mitigar sus dañinas consecuencias y garantizar un flujo de datos eficiente.

Una de las maneras efectivas de abordar el problema de la congestión en las redes es a través de la asignación adecuada de ancho de banda a los transmisores. En otras palabras, regular la velocidad a la que los dispositivos envían datos de modo que no saturen la red. Para lograr esto, es fundamental la colaboración entre las capas de red y de transporte.

La capa de red detecta la congestión, ya que es la única que puede proporcionar información directa sobre su estado. Sin embargo, capa de transporte es la causante de la congestión por ser la responsable de introducir los datos en la red. Por lo tanto, esta capa es responsable de reducir la carga que ofrece a la red cuando se detecta congestión.

Detección congestión



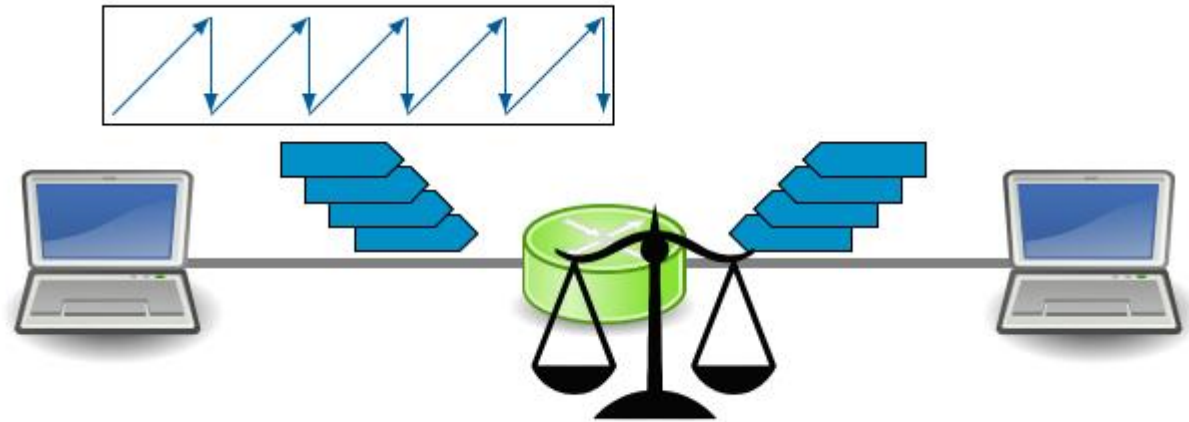
Fuente: Draw the OSI Model and list two functions of each layer <https://www.ques10.com/p/41437/draw-the-osi-model-and-list-two-functions-of-each/>

El Incremento Aditivo, Decremento Multiplicativo (Additive Increase, Multiplicative Decrease - AIMD) es una estrategia fundamental para la asignación de ancho de banda en redes de comunicación.

Alcanzar un equilibrio adecuado en la asignación de capacidad es un desafío debido a varios factores. Por un lado, el número de transmisores y la carga que cada uno ofrece varían constantemente. Por otro, la naturaleza distribuida de las redes significa que no hay una vista global del funcionamiento o estado de la red.

Bajo el modelo AIMD, los clientes incrementan aditivamente su tasa de transmisión en condiciones normales, es decir, cuando la red no presenta signos de congestión. Sin embargo, cuando se detecta congestión, los clientes responden disminuyendo su tasa de transmisión de manera multiplicativa. Este enfoque permite una reacción rápida y eficaz ante la congestión.

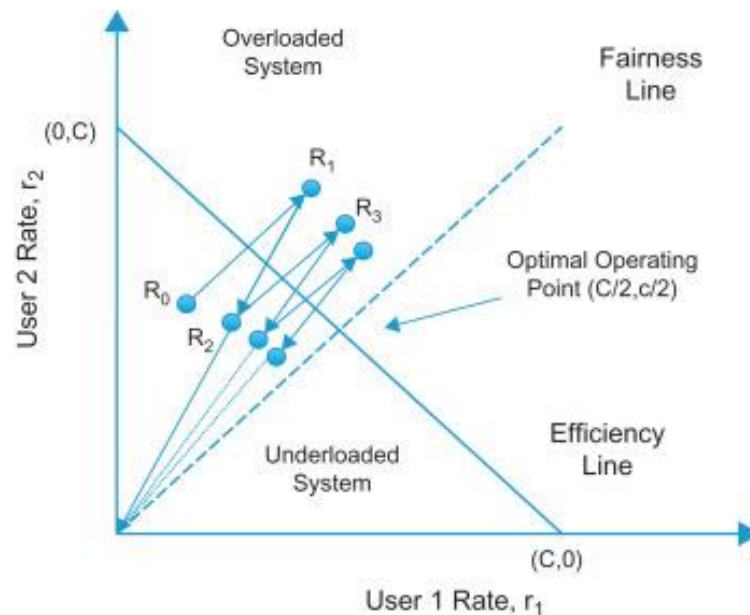
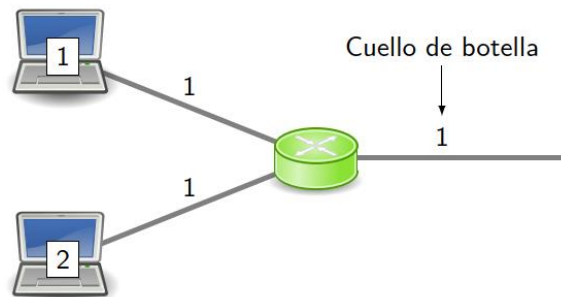
El modelo AIMD es el que inspira el control de congestión del protocolo TCP. Al equilibrar los incrementos y decrementos de la tasa de transmisión, AIMD ayuda a mantener la estabilidad de la red y a minimizar los problemas asociados a la congestión.



En un entorno de red, es común que varios clientes, como el cliente 1 y el cliente 2, compartan un cuello de botella en la capacidad de transmisión. Sin embargo, estos clientes no se comunican directamente entre sí. La ausencia de comunicación dificulta la coordinación en el uso del ancho de banda disponible. Para gestionar esta situación, es esencial contar con información sobre el estado de la red y la congestión.

La congestión se puede inferir de la pérdida de paquetes, un indicador claro de que la red está saturada y no puede manejar más tráfico en ese momento. Además, los enrutadores pueden proporcionar información explícita sobre la congestión. Esta información permite a los clientes ajustar su comportamiento de transmisión.

El modelo AIMD está diseñado para converger a la asignación óptima de ancho de banda, representada gráficamente como una forma de diente de sierra. Este patrón ilustra cómo los clientes incrementan su tasa de transmisión de manera gradual hasta que se detecta congestión, momento en el cual realizan una reducción drástica en su tasa.



Fuente: Internet Congestion Control
<https://doi.org/10.1016/B978-0-12-803583-2.00001-3>

AIMD 2/2

El modelo AIMD requiere únicamente información binaria sobre el estado de la red: saber si hay congestión o no en un momento dado. Esta información puede ser inferida desde los extremos de la conexión. Por ejemplo, a partir del aumento del retardo o de la pérdida de paquetes. Esto lo convierte en un sistema simple que no necesita cambios significativos en la infraestructura de la red. Una de las principales ventajas de AIMD es que no es necesario realizar modificaciones en los encaminadores de la red, lo que facilita su implementación en redes existentes.

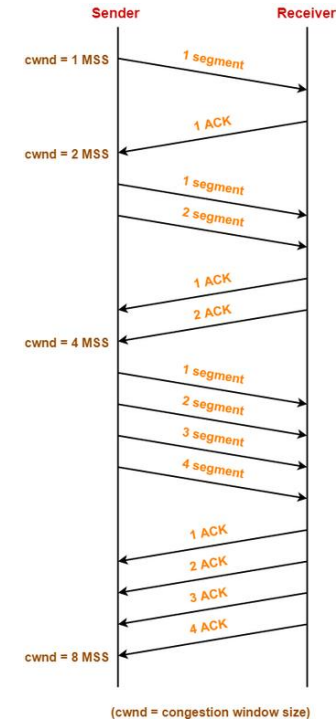
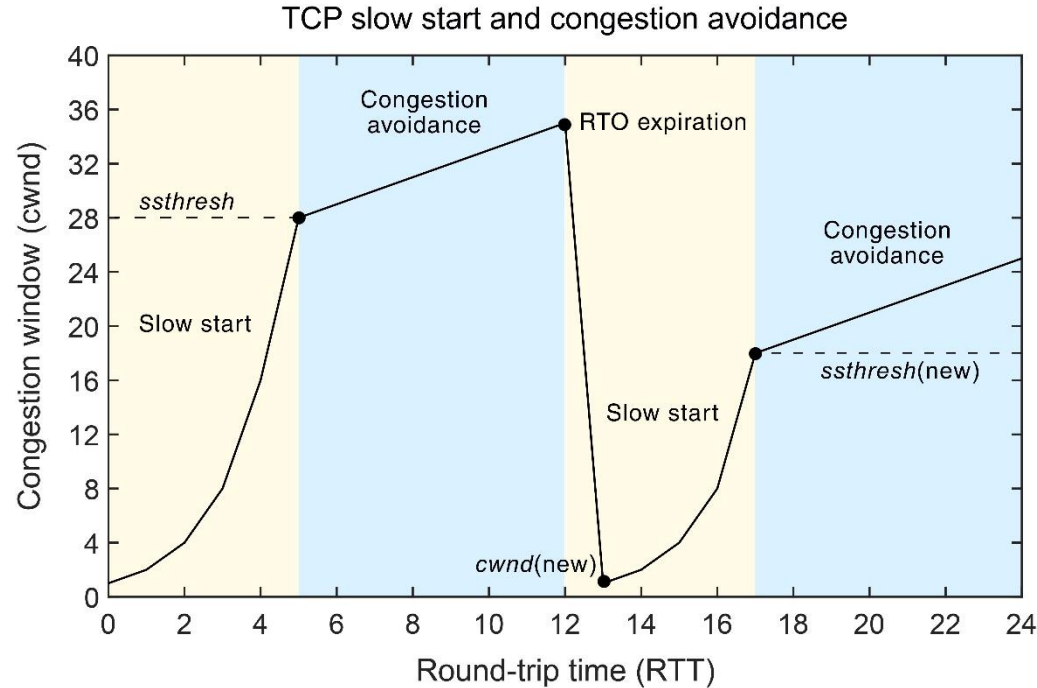
Una desventaja importante es que AIMD es una solución reactiva. La congestión solo se detecta una vez que ocurre, lo que puede resultar en una disminución temporal en la calidad del servicio hasta que se ajuste la tasa de transmisión. Los encaminadores pueden proporcionar información de congestión explícitamente. Se pueden marcar las cabeceras TCP de los paquetes por posible congestión. Aunque la solución preventiva es beneficiosa, requiere que los encaminadores de la red soporten este mecanismo, lo que podría implicar cambios en la infraestructura.

Control congestión TCP 1/2

La idea básica detrás del control de congestión en TCP se inspira en el modelo AIMD (Incremento Aditivo, Decremento Multiplicativo), que ajusta dinámicamente el tamaño de la ventana de congestión ($cwnd$). La ventana de congestión ($cwnd$) es el número de segmentos de datos pueden ser enviados sin recibir asentimiento del receptor. Representa el número de segmentos "en camino" en la red en un momento determinado. Se expresa en unidades de segmento máximo (Maximum Segment Size, MSS).

La tasa de transmisión de una conexión puede aproximarse mediante la fórmula $cwnd/RTT$ donde RTT es el tiempo de ida y vuelta (Round Trip Time, RTT). Si $cwnd$ es grande, la tasa de transmisión es alta y aumenta el riesgo de congestión. Si $cwnd$ es demasiado pequeña, se desaprovechan recursos.

La ventana de transmisión final en una conexión TCP se establece como el mínimo entre la ventana de congestión ($cwnd$) y la ventana de control de flujo. Esto asegura que el transmisor no envíe más datos de los que el receptor puede manejar, manteniendo así un flujo de datos eficiente y evitando la congestión.



El control de congestión en TCP (Protocolo de Control de Transmisión) es un aspecto fundamental para mantener la eficiencia y estabilidad de las conexiones en la red. Aquí se detallan los elementos clave y conceptos relacionados con este proceso:

Existen diversas implementaciones de TCP que abordan el control de congestión de manera diferente. Algunas de las más conocidas incluyen:

CUBIC: Utilizado en sistemas Linux, este algoritmo adapta la ventana de congestión de manera cúbica.

Compound TCP: Utilizado en Windows, combina señales de retardo y pérdida para ajustar la tasa de transmisión.

BBR (Bottleneck Bandwidth and Round-trip propagation time): Un algoritmo desarrollado por Google que utiliza estimaciones de ancho de banda y tiempo de ida y vuelta para optimizar la transmisión.

TCP Reno: Una de las versiones más antiguas, que emplea un mecanismo simple de detección de pérdida.

Control congestión TCP 2/2

algorithm	throughput	delay	Congestion window	reason
bbr	high	low	large	maximize the utilization of available bandwidth
Bbr wireless	high	high	large	considers bandwidth as another factor. tradeoff to get performance
Vegas	middle	low	small	It is pretty sensitive to RTT. Fairness.
Vegas wireless	middle	low	small	detects congestion by monitoring changes in RTT
Reno	high	low	small	Fast reduce, fairness.
reno wireless	low	high	small	High packet loss -> aggressive reduce
cubic	high	low	large	well-suited for high-speed, high-bandwidth networks
Cubic wireless	low	low	small	High packet loss -> aggressive reduce

¿PREGUNTAS?
andsuarez@uvigo.es



Salvo mención contraria, el contenido de este trabajo está bajo una licencia:

<http://creativecommons.org/licenses/by-sa/4.0/>